Computación Paralela y Distribuída

2026-II

José Fiestas PhD

11 de octubre de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 4: Comunicación y coordinación

Objetivos:

1. Pasos de Mensaje: MPI, Mensajes Punto a Punto, MPI, Comunicación Colectiva, Blocking vs non-blocking
2. Memoria Compartida: OMP, Constructores y cláusulas
3. Programacion Híbrida

Temática

OpenMP (Open Multi-processing)

¿Puede fallar el assert?

```
1  class C{
2      int x=0;
3      int y=0;
4      public:
5      void f(){
6      x=1; // A
7      y=1; // B
8      }
9      void g(){
10     int a=y; // C
11     int b=x; // D
12     assert(b>=a);
13     }
14     };
```

```
1  int main(){
2  int i=1;
3  while(true){
4      C c;
5      #pragma omp parallel
6      {
7          #pragma omp sections
8          {
9          #pragma omp section
10         c.f();
11         #pragma omp section
12         c.g();
13         }
14     }
15     if(++i%1000==0) cout<<i<<endl;
16     }
17 }
```

¿Puede fallar el assert?

Observamos distintas formas de ejecución (interleavings), como:
ABCD
ACBD
ACDB
CABD
CADB
CDAB
A siempre se ejecuta antes de B, C antes de D
Si asumimos 2 hilos y k sentencias, la lista tiene 2k elementos, y se necesita determinar k posiciones de un hilo (1 o 2) para determinar las formas de ejecución.
Esto equivale a una combinación sin repetición de 2k en k

$$\binom{2k}{k} = \frac{(2k)!}{k!^2} \sim \frac{4^k}{\sqrt{2k}}$$

¿Puede fallar el assert?

Demostrar **por contradicción** de que no hay formas de ejecución de f y g que puedan hacer que falle el assert()

Si $b < a$, entonces **a=1**, **b=0**
Pero si **a=1**, entonces **y=1** se ejecutó antes de **a=y**
si **b=0**, entonces **b=x** se ejecutó antes de **x=1**
Ya que cada hilo ejecuta las instrucciones secuencialmente,
a=y se ejecuta antes que **b=x**
x=1 se ejecuta antes que **y=1**

Por transitividad: **a=y** $\rightarrow$ **b=x** $\rightarrow$ **x=1** $\rightarrow$ **y=1**$\rightarrow$ **a=y**
Ciclos no son posibles en paralelo (**contradicción**)

Compilación (secuencial-paralelo)

Problema: compilador permite cambios que no afectan la semántica de un código **secuencial**. Una optimización durante la compilación (e.g. -O3) puede derivar en los siguientes códigos, con el mismo resultado en su versión secuencial (semánticamente equivalentes):

```
1  void f() {
2  x=1;
3  y=x+1;
4  z=x+1;
5  }
```

```
1  void f() {
2  x=1;
3  z=x+1;
4  y=x+1;
5  }
```

```
1  void f() {
2  x=1;
3  z=2;
4  y=2;
5  }
```

Considere una función g(){x=5;}. La funcion f() se asigna a un thread y g() a otro thread ¿Se obtendría en paralelo el mismo resultado?
En el segundo caso, x=1, z=y=2, y=6
En el tercer caso, y no puede ser 6

Compilación (secuencial-paralelo)

```
1  int x=0;
2  void wait(){
3  x=1;
4  while(x==1);
5  }
6  void arrive(){
7  x=2;
8  }
```

Ensamblador: x86

```
1  mov1    $0x1, x
2  test:
3  mov     x, %eax
4  cmp     $0x1,%eax
5  je      test
6
7  mov1    $0x2, x
```

Ensamblador: x86 optimizado (e.g. -O3)

```
1  mov1    $0x1, x
2  test:
3  jmp     test
4
5  mov1    $0x2, x
```

La optimización elimina las lineas 3 y 4 del ensamblador x86 no optimizado

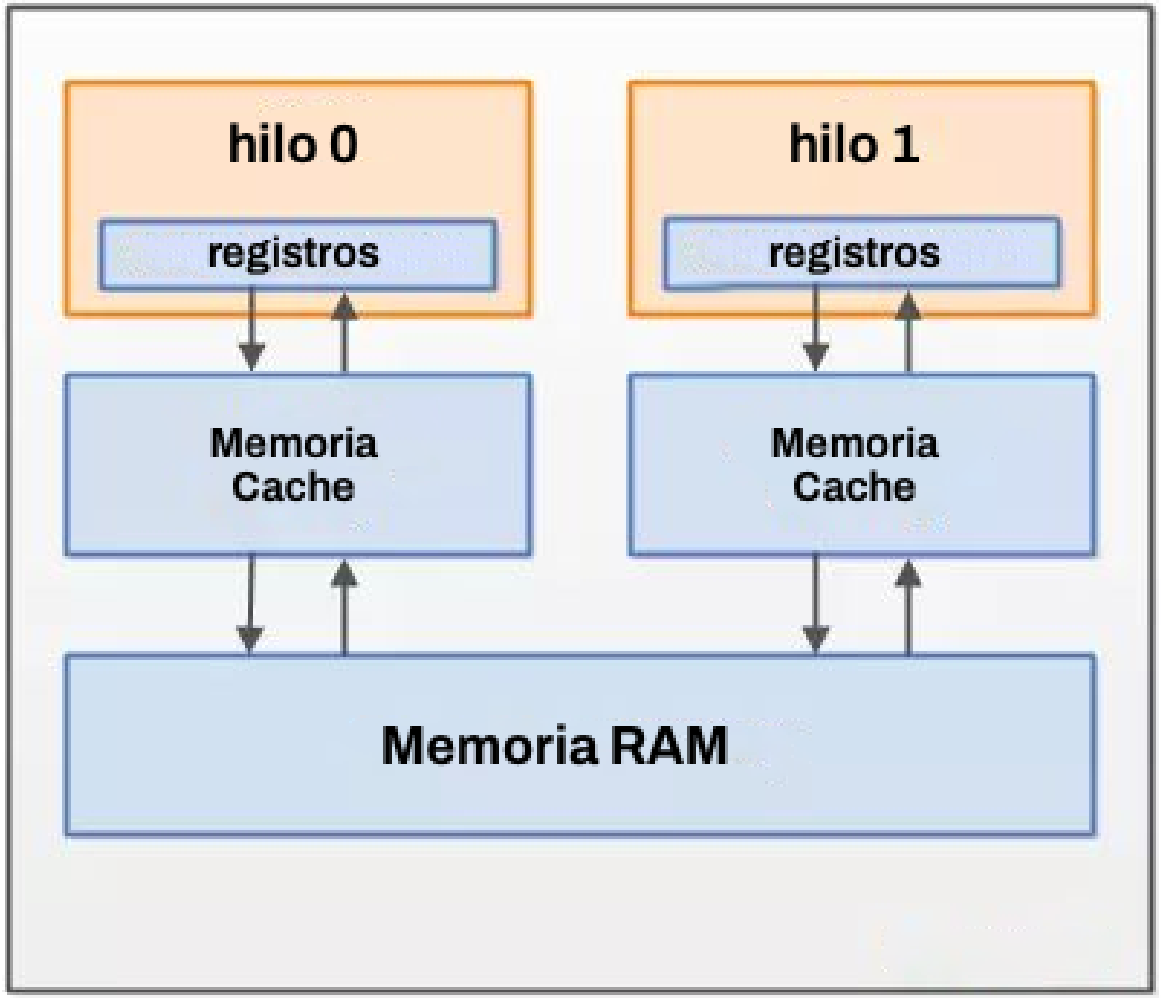
Compilación (secuencial-paralelo)

Compiladores modernos no garantizan un orden global de acceso a memoria, sino un orden local (por hilo). Logrando un factor de optimización típica de 3 - 10, básicamente a través de:

- Algunos eliminan para ello sentencias completas, en la búsqueda de performance.
- Además de ello, el procesador reordena instrucciones de acceso a memoria, también con el objetivo de mejorar performance
  - debido a la arquitectura pipeline, los hilos ejecutan múltiples instrucciones en forma simultánea, por lo que reordenan las instrucciones
  - cada hilo tiene un cache local, por lo que lectura y escritura en memoria compartida puede ser visible a otros hilos en distintas instancias (momentos)

Esto deja campo abierto a una re-optimización del código en paralelo para lograr mejoras

Arquitectura de memoria compartida



Modelo de memoria

El comportamiento de hilos en memoria compartida depende del hardware y software. Un modelo de memoria (e.g. en C++ https://en.cppreference.com/w/cpp/language/memory_model) garantiza operaciones en memoria correctos, bajo los siguientes criterios:

- **Orden del programa** (program order): dentro de un hilo existe un orden lógico entre las operaciones
- **Consistencia de memoria** (memory consistency): determina qué valores puede observar un hilo cuando otros realizan escrituras concurrentes.
- **Condición de carrera** (data race): cuando dos hilos acceden de manera conflictiva a la misma ubicación de memoria, al menos uno escribe y los accesos no están ordenados o sincronizados

Modelo de memoria

- Relación **happens-before**: relación de orden que permite establecer garantías entre operaciones. Si una *escritura* ocurre happens-before que una *lectura*, la lectura debe observar un valor compatible con ese orden. Los mecanismos de sincronización permiten establecer estas relaciones entre hilos.

OpenMP garantiza la consistencia utilizando los constructores discutidos en clase: *barrier, critical, atomic, task depend, reduction, flush*

Cuestionario

Join at

slido.com

#3941 398



Critical (repaso)

Regiones criticas solo permiten entrar un hilo a la vez

```
1  int i, sum=0, prod=1;
2  double t1, t2;
3  . . .
4  #pragma omp parallel default(shared)
5  {
6      int psum=0, pprod=1;
7      #pragma omp for
8      for(int i=0;i<ARR_SIZE;i++) {
9          psum+=a[i];
10         pprod*=a[i];
11     }
12     #pragma omp critical
13     sum+=psum;
14
15     #pragma omp critical
16     prod*=pprod;
17 }
```

Critical (repaso)

Se pueden nombrar secciones criticas independientes

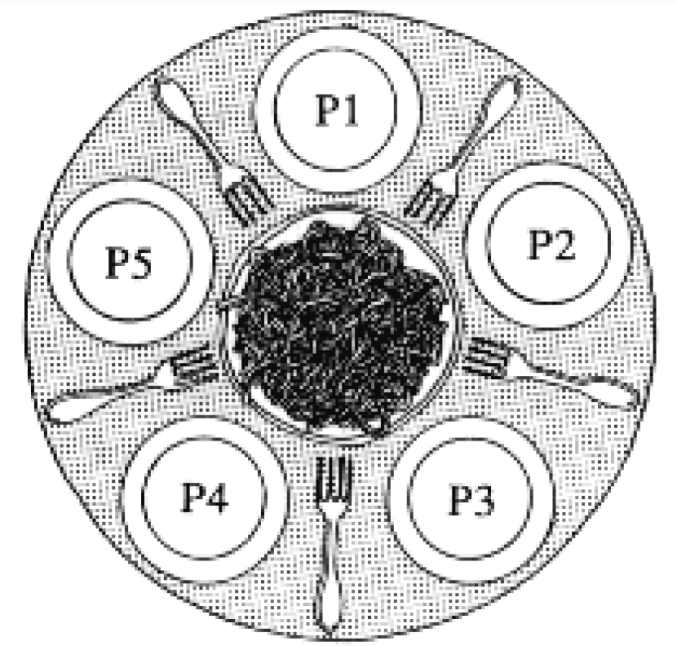
```
1  int i, sum=0, prod=1;
2  double t1, t2;
3  . . .
4  #pragma omp parallel default(shared)
5  {
6      int psum=0, pprod=1;
7      #pragma omp for
8      for(int i=0;i<ARR_SIZE;i++) {
9          psum+=a[i];
10         pprod*=a[i];
11     }
12     #pragma omp critical (section1)
13     {
14     for(int j=0;j<BIG_NUMBER;j++)
15     sum+=psum;
16     }
17     #pragma omp critical (section2)
18     {
19     for(int j=0;j<BIG_NUMBER;j++)
20     prod*=pprod;
21     }
22 }
```

Locks

Utilizando **locks** in OMP:
Un **lock** en OMP es un objeto (omp_lock_t) que puede ser poseído solo por un hilo a la vez. Posee 4 operaciones:
omp_init_lock(omp_lock_t *): inicializa el lock
omp_set_lock(omp_lock_t *): espera hast que el **lock** esté disponible, luego le asigna algún valor. Ningún otro hilo puede ser asignado hasta que está liberado.
omp_unset_lock(omp_lock_t *): libera el **lock**
omp_destroy_lock(omp_lock_t *): terminar el **lock**, lo contrario a init

Ejemplo: problema de los filósofos

N filósofos se disponen a cenar en una mesa redonda con N tenedores. Sin embargo, cada uno de ellos necesita dos tenedores para poder comer. Cada filósofo debe poder cenar, i.e. puede compartir sin problemas un tenedor con su vecino izquierdo así como con el derecho.



Ejemplo: problema de los filósofos

Algoritmo:

- piensa hasta que el tenedor izquierdo está disponible
- piensa hasta que el tenedor derecho está disponible
- come
- deja el tenedor izquierdo
- deja el tenedor derecho
- repite

Así se garantiza que algunos tengan dos tenedores, mientras que otros, ninguno.

Solución: locks

Solucion: locks

```
1  int main() {
2  int id,left,right;
3
4  // Inicializar los locks (tenedores)
5      for (int i = 0; i < N; ++i)
6          omp_init_lock(&forks[i]);
7
8      #pragma omp parallel num_threads(N) private(id,left,right)
9      {
10         id = omp_get_thread_num();
11         left = id;
12         right = (id + 1) % N;
13 }
```

Solución: locks

Solucion: locks

```
1     for (int i = 0; i < 30; ++i) {
2          think(id);
3  // los filosofos toman primero el tenedor izquierdo, luego el derecho
4      omp_set_lock(&forks[left]);
5      omp_set_lock(&forks[right]);
6      eat(id);
7      omp_unset_lock(&forks[left]);
8      omp_unset_lock(&forks[right]);
9     }
10     cout << "Filosofo " << id << " ha terminado.\n";
11    }
12    // Destruir los locks
13    for (int i = 0; i < N; ++i)
14        omp_destroy_lock(&forks[i]);
15
16     return 0;
```

Hay un problema de concurrencia en el código ¿Cómo lo solucionaría?

Locks

Calculo de histogramas (race condition)

```
1  #include<omp.h>
2  #include<stdio.h>
3  #define INP_SIZE (1<<26)
4  #define HIST_SIZE (1<<20)
5  int hist[HIST_SIZE];
6  int inp[INP_SIZE];
7  int main(){
8  int i, key, sum=0; double t1,t2;
9  // inicializar random input
10 t1= omp_get_wtime();
11 #pragma omp parallel for private (key)
12 for(int i=0; i<INP_SIZE; i++) {
13     key= inp[i];
14     hist[key]++;
15 }
16 t2=omp_get_wtime();
17 printf("Suma=%d ,tiempo %f \n", sum, t2-t1);
18 }
```

Locks

Calculo de histogramas. Solucion: critical (no es eficiente). ¿Se podrá usar secciones criticas?

```
1  #include<omp.h>
2  #include<stdio.h>
3  #define INP_SIZE (1<<26)
4  #define HIST_SIZE (1<<20)
5  int hist[HIST_SIZE];
6  int inp[INP_SIZE];
7  int main(){
8  int i, key, sum=0; double t1,t2;
9  // inicializar random input
10 t1= omp_get_wtime();
11 #pragma omp parallel for private (key)
12 for(int i=0; i<INP_SIZE; i++) {
13     key= inp[i];
14     #pragma omp critical
15     hist[key]++;
16 }
17 t2=omp_get_wtime();
18 printf("Suma=%d ,tiempo %f \n", sum, t2-t1);
19 }
```

Calculo de histogramas. Solucion: locks

```
1  omp_lock_t lock[HIST_SIZE]; // se define una estructura lock de tamanio HIS
2  int main(){
3  int i, key, sum=0; double t1,t2;
4  for(int i=0; i<HIST_SIZE; i++)
5      omp_init_lock(&(lock[i]));
6  t1= omp_get_wtime();
7  #pragma omp parallel for private (key)
8  for(int i=0; i<INP_SIZE; i++) {
9      key= inp[i];
10     omp_set_lock(&(lock[i]));
11     hist[key]++;
12     omp_unset_lock(&(lock[i]));
13 }
14 t2=omp_get_wtime();
15 for(int i=0; i<HIST_SIZE; i++)
16     omp_destroy_lock(&(lock[i]));
17 printf("Suma=%d ,tiempo %f \n", sum, t2-t1);
18 }
```

Bibliografía i

- https://www.openmp.org/
- Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP.* 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.
- Barbara Chapman, Gabriele Jost, and Ruud van der Pas: *Using OpenMP*. MIT Press, 2008, ISBN-13: 978-0-262-53302-7.
- Georg Hager, Gerhard Wellein: *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, 2010, 256p, ISBN13: 978-1-439-81192-4
- R.Chandra, L. Dagum, D. Kohr, D. Maydan, J. McDonald, and R. Menon: *Parallel programming in OpenMP*. Academic Press, San Diego, USA, 2000, ISBN 1-55860-671-8

UTEC
UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

